



Manual do Cluster Heisenberg

Guia completo para iniciantes e usuários avançados

Ilum – Escola de Ciência · CNPEM

Versão de agosto de 2026

Índice

1. Visão geral do cluster
 1. Arquitetura
 2. Os nós e seu hardware
 3. As filas (partições)
 4. Limites por usuário (QOS)
2. Acessando o cluster
 1. Pré-requisitos e conta
 2. SSH a partir de Linux ou macOS
 3. SSH a partir de Windows
 4. Login sem senha (chaves SSH)
 5. Reset de senha e chamados
3. Tutorial de Linux para iniciantes
 1. O que é o terminal
 2. Navegando pelo sistema de arquivos
 3. Manipulação de arquivos
 4. Processos e monitoramento
 5. Busca, filtros e pipes
 6. Dicas de produtividade
4. Transferindo arquivos
 1. WinSCP (Windows)
 2. scp e rsync (Linux/macOS)
5. Environment Modules

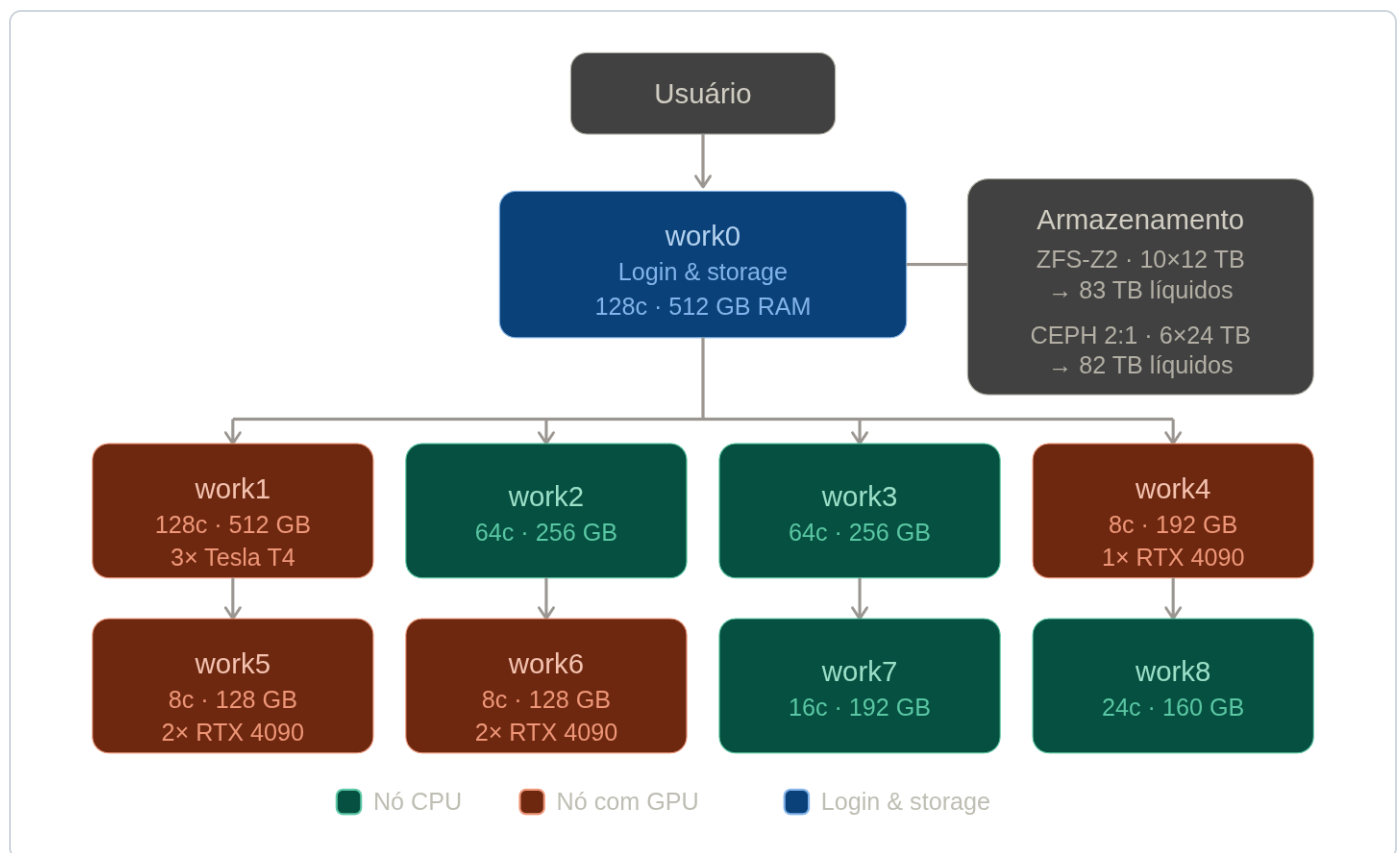
1. Comandos básicos
2. Aplicações científicas
3. Bibliotecas e compiladores
4. FDMNES
6. Python com Miniconda
 1. Instalando o Miniconda
 2. Criando o ambiente
 3. Instalando bibliotecas
 4. Ativando automaticamente
7. O Slurm: submetendo jobs
 1. Conceitos em 2 minutos
 2. Comandos essenciais
 3. Anatomia de um script
 4. Exemplos prontos
 5. Jobs de Python
 6. Sessão interativa
 7. Acompanhando e diagnosticando
 8. Variáveis de ambiente
 9. Memória: o erro mais comum
8. Regras de uso e monitoramento automático
 1. Todo cálculo passa pela fila
 2. Uso de GPU é monitorado
 3. Os e-mails da IlumA
 4. Um diretório por cálculo
9. Containers com Apptainer
 1. Por que Apptainer e não Docker
 2. Usando imagens prontas
 3. Containers com GPU
 4. Construindo sua própria imagem
 5. Containers dentro de jobs
10. Armazenamento e boas práticas
11. Perguntas frequentes
12. Guia rápido de referência
13. Glossário
14. Checklist do primeiro uso
15. Suporte

1. Visão geral do cluster

O Heisenberg é o cluster de computação científica da Ilum. Ele é formado por **10 máquinas (nós)**: o `work0`, chamado de *head node* ou nó de login, onde você entra e prepara seus cálculos, e os nós `work1` a `work9`, onde os cálculos de fato rodam.

Quem distribui os cálculos entre os nós é o **Slurm**, o gerenciador de filas — você nunca escolhe uma máquina "na mão": descreve os recursos de que precisa e o Slurm encontra onde rodar.

1.1 Arquitetura



Organização do cluster: você acessa o nó de login, que distribui os jobs aos nós de trabalho.

Todos os nós compartilham o mesmo `/home` e o mesmo `/opt` por rede. Na prática isso significa que **o arquivo que você criar no login aparece igual em qualquer nó** — você não precisa copiar nada entre máquinas.

1.2 Os nós e seu hardware

Nó	CPUs alocáveis (o que você pede no Slurm)	Cores físicos	Memória disponível ao Slurm	GPUs	Filas
work0 (login)	128	2 × 64	503 GB	—	cpu
work1	128	2 × 64	503 GB	3 × Tesla T4 (16 GB)	cpu, gpu
work2	64	2 × 32	251 GB	—	cpu
work3	64	2 × 32	251 GB	—	cpu
work4	16	1 × 16	124 GB	1 × RTX 4090 (24 GB)	gpu
work5	16	1 × 8 (2 threads/core)	124 GB	2 × RTX 4090 (24 GB)	gpu
work6	16	1 × 8 (2 threads/core)	124 GB	2 × RTX 4090 (24 GB)	gpu
work7	32	1 × 16 (2 threads/core)	187 GB	—	cpu
work8	48	2 × 12 (2 threads/core)	156 GB	1 × RTX 3060 (12 GB)	cpu, gpu
work9	128	1 × 64 (2 threads/core)	97 GB	—	cpu

! As duas colunas de CPU dizem coisas diferentes e as duas importam. **CPUs alocáveis** é o número que o Slurm entende quando você pede `--ntasks` ou `--cpus-per-task`. **Cores físicos** é o hardware real: nos nós com *2 threads por core*, pedir 16 CPUs no work5 entrega 8 cores de verdade. Para cálculo numérico pesado (VASP, GROMACS), o desempenho acompanha os cores físicos, não os threads.

⚠ **Atenção:** O `work9` tem muitas CPUs mas pouca memória disponível ao Slurm (97 GB para 128 CPUs). A diferença para a memória física da máquina é reserva para os serviços

internos do cluster. Se o seu cálculo usa muita RAM por processo, peça memória explicitamente (seção 7.9) para o Slurm não colocá-lo lá.

 **Dica:** Esta tabela é o estado planejado do cluster. Para saber o que está **disponível agora** — inclusive se algum nó está em manutenção — use `sinfo`. Nenhum manual acompanha o estado em tempo real.

1.3 As filas (partições)

Fila	Nós	CPUs totais	Para quê
cpu (padrão)	work0–work3, work7–work9	592	Cálculos de CPU: VASP, Quantum ESPRESSO, LAMMPS, scripts Python, etc.
gpu	work1, work4– work6, work8	224	Cálculos que usam GPU. Exige pedir a placa com <code>--gres=gpu:N</code> .

Se você não indicar fila, seu job entra na `cpu`. Não há limite de tempo imposto pela fila, mas **declare sempre um tempo estimado** com `--time` — jobs com tempo declarado são mais fáceis de encaixar e o escalonador os trata melhor (seção 7.3).

1.4 Limites por usuário (QOS)

Cada usuário tem um teto de CPUs simultâneas, definido pela sua QOS. Os tetos mais comuns são **8** e **16** CPUs; há perfis maiores para projetos específicos. Para saber o seu:

```
sacctmgr show assoc where user=$USER format=User,QOS%20
```

Se seus jobs ficarem pendentes com o motivo `QOSMaxCpuPerUserLimit` no `squeue`, você atingiu seu teto — eles entram conforme os anteriores terminam. Precisando de mais para um projeto específico, abra um chamado (seção 14).

2. Acessando o cluster

2.1 Pré-requisitos e conta

- Uma conta no cluster (a mesma do LDAP da Ilum). **Não tem?** Abra um chamado na Web Ilum: web-ilum.cnpem.br/chamado.
- Estar na rede da Ilum/CNPEM (presencialmente ou por VPN).
- Um terminal com SSH — já vem no Windows 10/11, macOS e Linux.

O endereço do cluster é:

```
heisenberg.cnpem.br      # preferível  
172.20.10.15             # mesmo destino, caso o nome não resolva
```

2.2 SSH a partir de Linux ou macOS iniciante

Abra o terminal e digite (trocando `fulana` pelo seu usuário):

```
ssh fulana@heisenberg.cnpem.br
```

Na primeira conexão o SSH pergunta se confia na máquina — responda `yes`. Depois digite sua senha; ela **não aparece** enquanto você digita, nem como asteriscos. Isso é normal.

```
(base) james@Heisenberg:~$ ssh james@heisenberg.cnpem.br
james@heisenberg.cnpem.br's password:

=====
 Bem vindos ao Cluster Heisenberg
=====

Em caso de problemas, abra chamado na Web Ilum:
https://web-ilum.cnpem.br/chamado
Para resetar sua senha entre em:
http://172.20.10.15
=====

Last login: Wed Jun 10 19:07:49 2026 from 10.0.105.10
```

Se aparecer este banner, você está dentro — no `work0`.

2.3 SSH a partir de Windows iniciante

O Windows 10/11 já vem com SSH. Abra o **PowerShell** (`Win` + `X` → "Terminal" ou "Windows PowerShell") e use o mesmo comando do Linux:

```
ssh fulana@heisenberg.cnpem.br
```

Se preferir uma interface gráfica com abas, terminal e transferência de arquivos juntos, o **MobaXterm** (mobaxterm.mobatek.net, edição Home, gratuita) é uma boa opção: *Session* → *SSH*, preencha *Remote host* = `heisenberg.cnpem.br` e o *username*.

2.4 Login sem senha (chaves SSH) avançado

Para não digitar a senha toda vez — e para scripts que copiam arquivos — crie um par de chaves na **sua máquina** e envie a parte pública ao cluster:

```
# na sua maquina (Linux, macOS ou PowerShell):
ssh-keygen -t ed25519          # Enter em tudo (ou defina uma passphrase)
ssh-copy-id fulana@heisenberg.cnpem.br

# no Windows sem ssh-copy-id:
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh fulana@heisenberg.cnpem.br "mkdir -p ~/.ssh;
```

2.5 Reset de senha e chamados

- Esqueceu ou quer trocar a senha: <http://172.20.10.15> no navegador.
- Qualquer problema, dúvida ou pedido: web-ilum.cnpem.br/chamado.

3. Tutorial de Linux para iniciantes iniciante

i Esta seção é para quem nunca usou um terminal. Se você já tem experiência com Linux, pule para a seção 5.

3.1 O que é o terminal

O terminal (também chamado de *shell* ou *linha de comando*) é uma interface de texto onde você digita comandos. No cluster, **toda interação é feita pelo terminal** — não há área de trabalho gráfica.

Após o login você verá algo como:

```
fulana@work0:~$
```

Isso significa: `fulana` = seu usuário; `work0` = a máquina onde você está; `~` = você está na sua pasta pessoal (*home*); `$` = pronto para receber comandos.

3.2 Navegando pelo sistema de arquivos

O Linux organiza arquivos em uma árvore de diretórios cuja raiz é `/`. Sua pasta pessoal fica em `/home/seu_usuario` e pode ser referenciada pelo atalho `~`.

Saber onde você está

```
$ pwd
/home/fulana
```

`pwd` (*print working directory*) mostra o diretório atual.

Listar arquivos

```
$ ls
dados/  scripts/  simulacao.sh

$ ls -lh
```

```
total 12K
drwxr-xr-x 2 fulana fulana 4096 jun  1 10:00 dados
drwxr-xr-x 2 fulana fulana 4096 jun  1 10:01 scripts
-rw-r--r-- 1 fulana fulana  512 jun  3 14:22 simulacao.sh
```

`-l` exibe detalhes (permissões, tamanho, data); `-h` exibe tamanhos legíveis (KB, MB).

Entrar em um diretório

```
$ cd dados
$ pwd
/home/fulana/dados

$ cd ..      # volta um nivel acima
$ cd ~      # volta para a home
$ cd -      # volta para o diretorio anterior
```

Criar diretórios

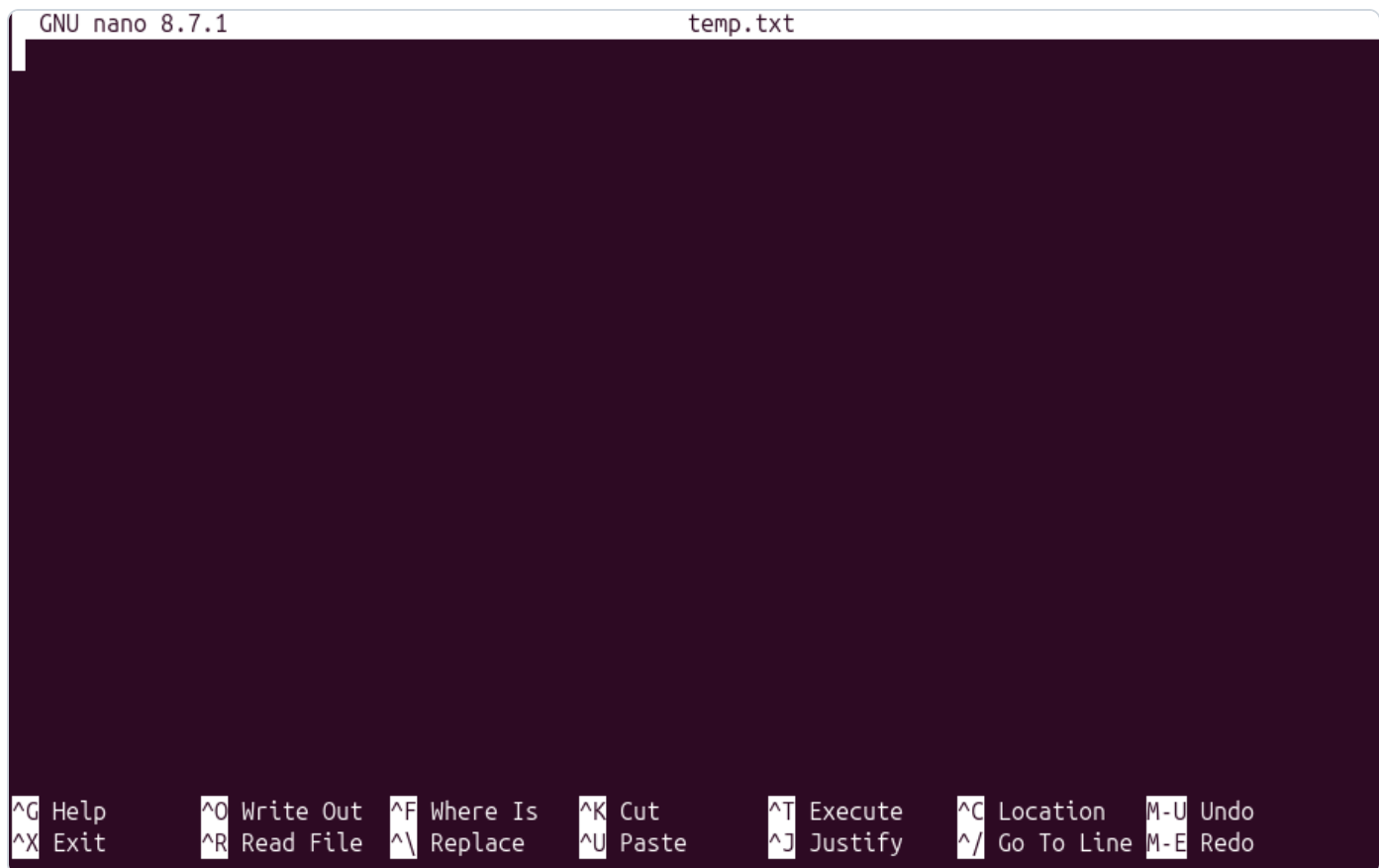
```
$ mkdir meu_projeto
$ mkdir -p meu_projeto/resultados/graficos  # cria toda a arvore de uma vez
```

3.3 Manipulação de arquivos

Criar e editar um arquivo de texto

```
$ nano meu_arquivo.txt
```

O `nano` é um editor simples que funciona direto no terminal.



O editor `nano` : os atalhos aparecem no rodapé, onde `^` significa `Ctrl`.

- `Ctrl+O` → salvar (*O de Output*)
- `Ctrl+X` → sair
- `Ctrl+W` → buscar texto

Visualizar conteúdo

```
$ cat arquivo.txt           # exibe o arquivo inteiro
$ less arquivo.txt          # paginado (q para sair, /termo para buscar)
$ head -20 arquivo.txt      # primeiras 20 linhas
$ tail -20 arquivo.txt       # ultimas 20 linhas
$ tail -f slurm-12345.out    # acompanha o arquivo em tempo real
```

Copiar, mover e remover

```
$ cp arquivo.txt copia.txt    # copiar arquivo
$ cp -r pasta/ pasta_backup/  # copiar pasta inteira
$ mv arquivo.txt novo_nome.txt # renomear
$ mv arquivo.txt /home/fulana/dados/ # mover para outra pasta
```

```
$ rm arquivo.txt          # remover arquivo
$ rm -r pasta/            # remover pasta e tudo dentro
```

👉 **Importante:** O comando `rm` apaga arquivos **permanentemente** no Linux — não existe lixeira no terminal. Use com muito cuidado, especialmente com `-r` (recursivo). Nunca execute `rm -rf /` ou `rm -rf *` sem ter certeza absoluta do que está fazendo.

3.4 Processos e monitoramento

```
$ top                    # monitor de processos em tempo real (q para sair)
$ htop                  # versao melhorada do top
$ ps aux | grep $USER   # listar seus processos
$ kill 12345            # encerrar o processo de PID 12345
```

⚠ **Atenção:** Estes comandos servem para **observar**. Não use o nó de login para rodar cálculo: veja a seção [8.1](#), que é regra do cluster e é verificada automaticamente.

3.5 Busca, filtros e pipes

```
$ grep "erro" log.out    # busca a palavra "erro" no arquivo
$ grep -r "funcao" scripts/ # busca recursiva em uma pasta
$ find . -name "*.py"    # encontra arquivos .py a partir daqui
$ wc -l arquivo.txt      # conta as linhas de um arquivo

$ ls -lh > lista.txt      # redireciona a saida para um arquivo (sobrescreve)
$ ls -lh >> lista.txt     # redireciona acrescentando ao final
$ cat log.out | grep "ERRO" # filtra do log so as linhas com "ERRO"
$ sort numeros.txt | uniq # ordena e remove duplicatas
```

3.6 Dicas de produtividade

💡 **Dica:**

- `Tab` — completa automaticamente nomes de arquivos e comandos.
- `↑` / `↓` — navega pelo histórico de comandos.

- `PageUp` / `PageDown` — busca no histórico pelo que já foi digitado: escreva `sr` e pressione `PageUp` para ver o último comando que começava assim.
- `Ctrl`+`C` — cancela o comando em execução.
- `Ctrl`+`L` — limpa a tela.
- `Ctrl`+`A` / `Ctrl`+`E` — vai para o início/fim da linha.
- `history` — lista os últimos comandos digitados.
- `man ls` — manual de qualquer comando (`q` para sair).

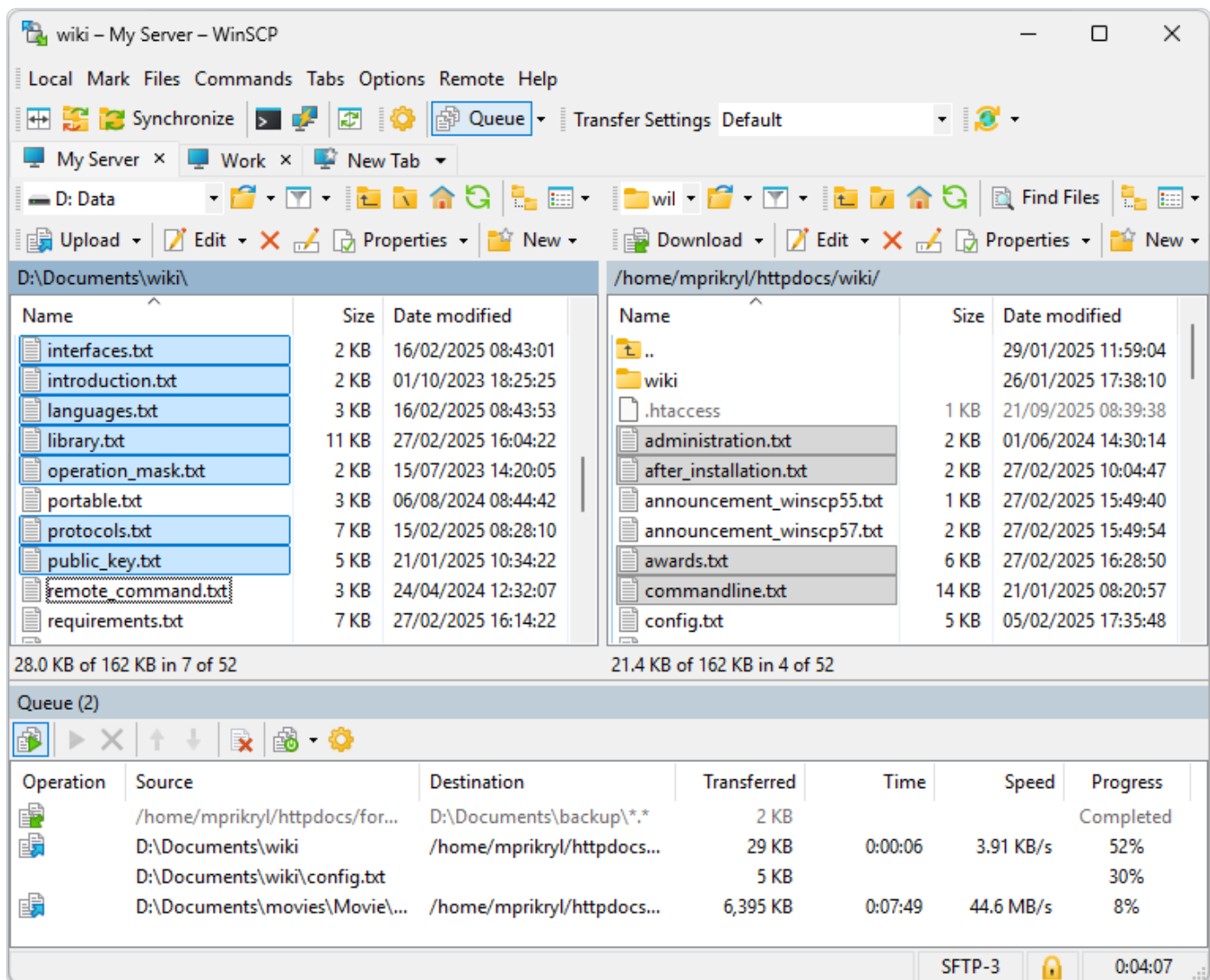
4. Transferindo arquivos

i Os comandos desta seção são digitados no terminal do **seu próprio computador**, não no cluster.

4.1 WinSCP (Windows) iniciante

O WinSCP mostra seus arquivos locais de um lado e os do cluster do outro, e você arrasta entre eles.

1. Baixe em winscp.net/eng/download.php.
2. Execute o instalador e aceite as opções padrão (*Typical installation*); quando perguntar o estilo da interface, *Commander* é o de dois painéis.
3. Na janela *Login*, preencha:
 - **File protocol:** SFTP
 - **Host name:** `heisenberg.cnpem.br`
 - **Port number:** 22
 - **User name / Password:** os da sua conta
4. Clique *Save* (para não preencher de novo) e depois *Login*.
5. Painel esquerdo = seu computador; painel direito = seu home no cluster. **F5** copia, **F6** move.



WinSCP em modo Commander: local à esquerda, cluster à direita.

Dica: Você pode editar um arquivo do cluster com duplo clique — o WinSCP baixa, abre no editor e reenvia ao salvar. Prático para ajustar um script de submissão sem usar o nano .

4.2 scp e rsync (Linux/macOS) iniciante

```
# enviar um arquivo para o cluster
scp resultado.dat fulana@heisenberg.cnpem.br:~/projeto/

# trazer um diretorio inteiro do cluster
scp -r fulana@heisenberg.cnpem.br:~/projeto/saidas/ .
```

```
# rsync: retoma de onde parou e so copia o que mudou (melhor p/ muita coisa)
rsync -av --progress dados/ fulana@heisenberg.cnpem.br:~/projeto/dados/
```

 **Dica:** Para muitos arquivos pequenos, compacte antes (`tar czf dados.tar.gz dados/`): transferir um arquivo grande é muito mais rápido do que milhares de pequenos.

5. Environment Modules

Os softwares científicos ficam em `/opt/apps` e são carregados com o *Environment Modules* — cada `module load` configura PATH, bibliotecas e variáveis daquele programa, sem conflitar com os demais.

5.1 Comandos básicos

```
module avail           # lista tudo que existe
module load gromacs/2026.0 # carrega (use sempre no script de submissao)
module list            # o que esta carregado agora
module show gromacs/2026.0 # mostra o que o modulo altera
module unload gromacs/2026.0 # descarrega
module purge           # descarrega tudo
```

 **Dica:** Carregue os módulos **dentro do script de submissão**, não no `.bashrc`. Assim cada job documenta exatamente as versões que usou — essencial para reproduzir resultados depois.

5.2 Aplicações científicas

Programa	Versões (module)	Área
VASP	6.2.0, 6.2.0-aocl, 6.6.1-aocl	DFT / materiais (licenciado)
Quantum ESPRESSO	7.5, 7.6	DFT / materiais
GROMACS	2026.0	Dinâmica molecular (com CUDA)
AMBER	2024	Dinâmica molecular biomolecular
ORCA	6.1.1	Química quântica
LAMMPS	<code>/opt/apps/lammps</code>	Dinâmica molecular clássica
NAMD	<code>/opt/apps/NAMD</code>	Dinâmica molecular

AMS	/opt/apps/AMS	Amsterdam Modeling Suite (licenciado)
PLUMED	/opt/apps/plumed (e gromacs-plumed)	Metadinâmica / variáveis coletivas
FDMNES	/opt/apps/FDMNES	Espectroscopia de raios X (XANES/XES)
Ollama	/opt/apps/ollama	Modelos de linguagem locais

5.3 Bibliotecas e compiladores

avançado

Módulo	Versões	Observação
openmpi	4.1.8, 5.0.9	MPI padrão do cluster (com UCX e PMIx)
intel	oneAPI 2025.3	compiladores (icx/ix), MKL, Intel MPI, TBB, DNNL
aocl	5.3.0 (mt/st)	AMD Optimizing Libraries (BLAS/LAPACK/FFTW)
cuda	12.4, 13.1	toolkit CUDA para compilar código de GPU
scalapack	2.2.0 (ompi 5.0.9)	álgebra linear distribuída
elpa	2024.05, 2026.02	autovalores para DFT
ucx / pmix / hwloc	1.21.0 / 5.0.9 / 2.12.2	infraestrutura de comunicação MPI

Compiladores do sistema: GCC 13.3 (gcc , gfortran), sempre disponíveis, sem módulo.

5.4 FDMNES

O FDMNES (espectros XANES/XES) está em /opt/apps/FDMNES com runtime MPI próprio embutido — **não** precisa de module load . Use o wrapper mpirun_fdmnes :

```
#!/bin/bash
#SBATCH --job-name=fdmnes
#SBATCH --partition=cpu
#SBATCH --ntasks=16
```

```
#SBATCH --mem=32G
#SBATCH --time=08:00:00

# 16 processos, cada um calculando uma energia
/opt/apps/FDMNES/mpirun_fdmnes -np $SLURM_NTASKS

# variante: grupos de 4 processos por energia (solver MUMPS paralelo)
# HOST_NUM_FOR_MUMPS=4 /opt/apps/FDMNES/mpirun_fdmnes -np $SLURM_NTASKS
```

O arquivo de controle `fdmfile.txt` deve estar no diretório de submissão, como de costume no FDMNES.

6. Python com Miniconda

Cada usuário instala o seu próprio Miniconda na sua home. Isso lhe dá qualquer versão de Python e qualquer biblioteca, sem depender do administrador e sem conflitar com os outros usuários.

6.1 Instalando o Miniconda iniciante

Logado no cluster, rode:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
```

O instalador vai perguntar três coisas:

1. A licença — segure `Enter` (ou `q` para pular) e digite `yes`.
2. O local — aceite o padrão (`/home/SEU_USUARIO/miniconda3`) com `Enter`.
3. *"Do you wish to update your shell profile..."* — responda `yes`.

Depois, **saia e entre de novo no cluster** (ou rode `source ~/.bashrc`). O prompt deve passar a mostrar `(base)` no início da linha. Teste:

```
conda --version
python --version
```

 **Dica:** Se o `conda` não for encontrado depois disso, você provavelmente respondeu "no" à terceira pergunta. Rode `~/miniconda3/bin/conda init bash`, saia e entre de novo.

6.2 Criando o ambiente iniciante

Nunca trabalhe no ambiente `base`: crie um por projeto, para que quebre um não quebre os outros. Por convenção na Ilum o ambiente principal se chama `ilumpy`, mas o nome é livre.

```
# criar
conda create -n ilumpy python=3.11
```

```
# ativar (faça sempre antes de usar)
conda activate ilumpy

# o prompt passa a mostrar o ambiente ativo:
# (ilumpy) fulana@work0:~$

conda deactivate      # desativar
conda env list         # listar seus ambientes
conda remove -n ilumpy --all  # apagar um ambiente
```

6.3 Instalando bibliotecas

```
conda activate ilumpy

# com pip (funciona para praticamente tudo do PyPI)
pip install numpy scipy matplotlib pandas ase pymatgen jupyter

# com conda (melhor para pacotes com dependências compiladas complexas)
conda install -c conda-forge rdkit
```

 **Dica:** Regra prática: use `pip` por padrão; recorra ao `conda install -c conda-forge` quando o `pip` falhar ao compilar alguma dependência. Evite misturar os dois para o *mesmo* pacote.

Para PyTorch com GPU (fila `gpu`):

```
pip install torch --index-url https://download.pytorch.org/whl/cu124
```

6.4 Ativando o ambiente automaticamente

Se você usa sempre o mesmo ambiente, acrescente ao final do `~/.bashrc`:

```
conda activate ilumpy
```

⚠ **Atenção:** Isso vale para o seu shell interativo. **Dentro de um script de job, ative o ambiente explicitamente** — o job não lê o seu `.bashrc` da mesma forma:

```
source ~/miniconda3/etc/profile.d/conda.sh  
conda activate ilumpy
```

7. O Slurm: submetendo jobs

7.1 Conceitos em 2 minutos iniciante

- **Job:** um pedido de cálculo — um script mais a descrição dos recursos (CPUs, memória, tempo, GPUs).
- **Fila (partição):** o grupo de nós onde o job pode rodar (`cpu` ou `gpu`).
- **Tarefa (`--ntasks`):** um processo. Um programa MPI com 32 ranks = 32 tarefas. Um script Python comum = 1 tarefa.
- **CPUs por tarefa (`--cpus-per-task`):** threads que cada processo usa (OpenMP, numpy multithread).

Você escreve um script com diretivas `#SBATCH` , entrega com `sbatch` , e o Slurm o executa assim que houver recursos, gravando a saída num arquivo `slurm-JOBID.out` no diretório de onde você submeteu.

7.2 Comandos essenciais

Comando	O que faz
<code>sbatch script.sh</code>	Submete o job. Devolve o número (JOBID).
<code>squeue --me</code>	Mostra seus jobs: rodando (R) ou pendentes (PD) e por quê.
<code>squeue</code>	A fila inteira, de todos.
<code>scancel JOBID</code>	Cancela um job seu (na fila ou rodando).
<code>sinfo</code>	Estado das filas e nós (idle, mix, alloc, down).
<code>sacct -j JOBID</code>	Histórico: estado final, tempo, memória usada.
<code>scontrol show job JOBID</code>	Todos os detalhes de um job.

7.3 Anatomia de um script de submissão

```
#!/bin/bash
#SBATCH --job-name=meu-calculo    # aparece no squeue
```

```
#SBATCH --partition=cpu          # fila: cpu ou gpu
#SBATCH --ntasks=32              # numero de tarefas (processos MPI)
#SBATCH --cpus-per-task=1        # CPUs por tarefa (threads)
#SBATCH --mem=120G               # memoria TOTAL do job no no
#SBATCH --time=1-12:00:00        # tempo max: 1 dia e 12 h (D-HH:MM:SS)
#SBATCH --output=saida_%j.out    # %j vira o numero do job
#SBATCH --error=erro_%j.err

# ... aqui vem o que executar ...
```

Principais parâmetros

Longo	Curto	O que define
<code>--job-name=nome</code>	<code>-J</code>	Nome do job no <code>squeue</code>
<code>--partition=cpu</code>	<code>-p</code>	Fila
<code>--ntasks=N</code>	<code>-n</code>	Número de tarefas (processos)
<code>--cpus-per-task=N</code>	<code>-c</code>	Threads por tarefa
<code>--mem=64G</code>	—	Memória total no nó
<code>--time=HH:MM:SS</code>	<code>-t</code>	Tempo máximo estimado
<code>--gres=gpu:1</code>	—	Quantas GPUs reservar
<code>--output=arq_%j.out</code>	<code>-o</code>	Arquivo de saída
<code>--mail-type=END, FAIL</code>	—	Avisa por e-mail ao terminar/falhar

As formas longa e curta são equivalentes. Este manual usa a longa por ser mais legível; use a que preferir, mas não misture as duas no mesmo script.

As duas diretivas que **todo** job deveria ter, além de fila e tarefas:

- `--time` : sem ela o job fica sem limite, o que atrapalha o escalonador e impede o *backfill* (encaixar jobs curtos nos buracos da fila). Estime com folga — o job termina quando acabar, não espera o prazo.
- `--mem` : o padrão é **1 GB por CPU alocada**, pouco para a maioria dos cálculos. Se o job passar do que pediu, é morto por falta de memória (seção [7.9](#)).

! O cluster aloca **um nó por job**. Não use `--nodes` : peça as tarefas com `--ntasks` e o Slurm as acomoda no nó adequado.

7.4 Exemplos prontos

Serial (1 processo)

```
#!/bin/bash
#SBATCH --job-name=serial
#SBATCH --partition=cpu
#SBATCH --ntasks=1
#SBATCH --mem=8G
#SBATCH --time=04:00:00

./meu_programa entrada.in > saida.out
```

MPI (múltiplos processos)

```
#!/bin/bash
#SBATCH --job-name=mpi
#SBATCH --partition=cpu
#SBATCH --ntasks=32
#SBATCH --mem=64G
#SBATCH --time=24:00:00

module load openmpi/5.0.9
srun ./meu_programa_mpi
```



Dica: Dentro de um job, prefira `srun programa` a `mpirun -np N programa`: o `srun` herda do Slurm o número de tarefas e a distribuição, sem risco de divergência entre o que você pediu e o que o MPI lança.

VASP

```
#!/bin/bash
#SBATCH --job-name=vasp_dft
#SBATCH --partition=cpu
#SBATCH --ntasks=32
#SBATCH --mem=120G
```

```
#SBATCH --time=3-00:00:00
#SBATCH --output=vasp_%j.out
#SBATCH --error=vasp_%j.err

module load vasp/6.6.1-aocl
srun vasp_std
```

⚠ **Atenção:** O VASP instalado aqui é compilado **sem OpenMP**. Definir `OMP_NUM_THREADS` não tem efeito: o paralelismo é MPI puro, ajustado pelos parâmetros `NCORE` e `KPAR` do `INCAR`.

GROMACS (exemplo completo)

```
#!/bin/bash
#SBATCH --job-name=gromacs_md
#SBATCH --partition=cpu
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=32
#SBATCH --mem=64G
#SBATCH --time=48:00:00
#SBATCH --output=gromacs_%j.out
#SBATCH --error=gromacs_%j.err
#SBATCH --mail-type=END,FAIL
#SBATCH --mail-user=meu@email.com.br

module load gromacs/2026.0
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK

# Equilibracao
gmx mdrun -v -deffnm equilibracao -ntmpi 1 -ntomp $OMP_NUM_THREADS

# Producao
gmx mdrun -v -deffnm producao -ntmpi 1 -ntomp $OMP_NUM_THREADS
```

GPU (treinamento PyTorch)

```
#!/bin/bash
#SBATCH --job-name=treino
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1          # quantas GPUs (gpu:2 p/ duas)
#SBATCH --ntasks=1
```

```
#SBATCH --cpus-per-task=8          # CPUs p/ dataloader etc.
#SBATCH --mem=32G
#SBATCH --time=12:00:00

source ~/miniconda3/etc/profile.d/conda.sh
conda activate ilumpy
python -u treino.py
```

Para exigir um modelo específico de GPU: `--gres=gpu:4090:1` , `--gres=gpu:t4:1` ou `--gres=gpu:3060:1` .

Array — varrer muitos casos avançado

```
#!/bin/bash
#SBATCH --job-name=varredura
#SBATCH --partition=cpu
#SBATCH --ntasks=1
#SBATCH --mem=4G
#SBATCH --time=01:00:00
#SBATCH --array=0-99%10          # 100 casos, no maximo 10 simultaneos

python -u simula.py --caso $SLURM_ARRAY_TASK_ID
```

7.5 Jobs de Python

Um script Python comum é **serial**: por causa do GIL, ele usa uma CPU só, por mais que você peça. Pedir 32 CPUs para um script assim reserva 32 e usa 1 — e o cluster vai avisar você (seção 8.3).

Python serial

```
#!/bin/bash
#SBATCH --job-name=analise
#SBATCH --partition=cpu
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=8G
#SBATCH --time=02:00:00

source ~/miniconda3/etc/profile.d/conda.sh
```

```
conda activate ilumpy
python -u analise.py
```

 **Dica:** O `-u` desliga o buffer de saída: sem ele, os `print()` só aparecem no arquivo quando o script termina, e o `tail -f` não mostra nada enquanto isso.

Python paralelo (multiprocessing)

Para de fato usar várias CPUs, o script precisa paralelizar explicitamente — com `multiprocessing`, `joblib`, `dask`, ou bibliotecas que já usam BLAS multithread (numpy, scipy).

```
#!/bin/bash
#SBATCH --job-name=paralelo
#SBATCH --partition=cpu
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=32G
#SBATCH --time=06:00:00

source ~/miniconda3/etc/profile.d/conda.sh
conda activate ilumpy

# faz o numpy/scipy respeitarem o que o Slurm alocou
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export MKL_NUM_THREADS=$SLURM_CPUS_PER_TASK

python -u simulacao.py
```

```
# dentro do simulacao.py: usar exatamente as CPUs alocadas
import os
from multiprocessing import Pool

n = int(os.environ.get("SLURM_CPUS_PER_TASK", 1))
with Pool(n) as p:
    resultados = p.map(minha_funcao, lista_de_casos)
```

7.6 Sessão interativa

Para testar algo com os recursos de um nó de cálculo, num shell de verdade:

```
# shell com 8 CPUs e 16G por ate 2 h
srun --partition=cpu --ntasks=1 --cpus-per-task=8 --mem=16G --time=02:00:00 --pty bash

# com uma GPU
srun --partition=gpu --gres=gpu:1 --cpus-per-task=4 --mem=16G --time=01:00:00 --pty bash
```

Ao sair do shell (`exit`), o job termina e os recursos são liberados. É a forma correta de "entrar num nó" — nunca por `ssh` .

7.7 Acompanhando e diagnosticando

```
# por que meu job esta pendente?
squeue --me          # veja a coluna NODELIST(REASON)
#   Resources        = aguardando maquina livre (normal)
#   Priority          = ha jobs na frente
#   QOSMaxCpuPerUserLimit = voce atingiu seu teto de CPUs (secao 1.4)

# acompanhar a saida em tempo real
tail -f slurm-JOBID.out

# depois que terminou: quanto usou de fato?
sacct -j JOBID -o JobID,State,Elapsed,TotalCPU,MaxRSS,ReqMem
#   MaxRSS    = pico de memoria POR TAREFA
#   TotalCPU vs Elapsed x CPUs = eficiencia
```

7.8 Variáveis de ambiente do Slurm

Variável	Conteúdo
<code>\$SLURM_JOB_ID</code>	ID do job
<code>\$SLURM_JOB_NAME</code>	Nome do job
<code>\$SLURM_NTASKS</code>	Número total de tarefas
<code>\$SLURM_CPUS_PER_TASK</code>	CPUs por tarefa
<code>\$SLURM_MEM_PER_NODE</code>	Memória por nó (MB)
<code>\$SLURM_NODELIST</code>	Nós alocados

<code>\$SLURM_SUBMIT_DIR</code>	Diretório de onde o <code>sbatch</code> foi chamado
<code>\$SLURM_ARRAY_TASK_ID</code>	Índice do caso, em jobs de array

7.9 Memória: o erro mais comum

Os dois sintomas clássicos:

- **Job morto com `OUT_OF_MEMORY`** : pediu menos do que usa. O limite é aplicado de verdade pelo cgroup — passou, morreu. Aumente o `--mem` em ~50% e resubmeta.
- **Job espera demais na fila ou "não cabe"**: pediu além do necessário, ou além do que qualquer nó tem. Memória reservada e não usada fica bloqueada para os outros.

Para calibrar, rode uma vez com folga e consulte o `MaxRSS` no `sacct` . Lembre que `MaxRSS` é **por tarefa**: num job MPI o total é aproximadamente `MaxRSS` × tarefas no mesmo nó — e essa soma *superestima*, porque páginas de memória compartilhadas entre os processos são contadas repetidas.



Dica: Os relatórios automáticos da IllumA (seção [8.3](#)) já fazem essa conta e sugerem o `--mem` adequado para a próxima submissão.

8. Regras de uso e monitoramento automático

O cluster verifica sozinho, a cada 10 minutos, se os recursos estão sendo usados conforme reservados. As regras abaixo não são recomendações: são aplicadas automaticamente.

8.1 Todo cálculo passa pela fila

 **Importante:** Rodar cálculos diretamente nos nós — entrando por SSH e executando na mão, fora do Slurm — **não é permitido**. Processos de cálculo fora da fila são finalizados automaticamente e o usuário é notificado por e-mail.

O motivo é justiça: o que roda por fora não reserva recursos e rouba CPU e memória de quem esperou na fila. No `work0` (login) valem as tarefas leves — editar, compilar, instalar pacotes, testes de poucos segundos. Para qualquer coisa maior, use `sbatch` ou uma sessão interativa (seção 7.6).

8.2 Uso de GPU é monitorado

GPUs são o recurso mais disputado. O monitor confere se os jobs na fila `gpu` estão de fato usando as placas reservadas:

- **Job sem usar GPU nenhuma:** você recebe avisos por e-mail; persistindo, o job é cancelado.
- **Job usando só parte das GPUs** (reservou 2, usa 1): recebe dois avisos ao longo de cerca de uma hora e, se nada mudar, é cancelado. A tolerância existe para cálculos que fazem um pré-processamento longo numa placa só.

Reserve apenas o que vai usar. Precisa da fila `gpu` só pelas CPUs? Não faça isso — use a fila `cpu`.

8.3 Os e-mails da IlumA

A IlumA é a assistente automática do cluster. Ela analisa seus jobs e envia dois tipos de mensagem:

- **Na submissão:** conselhos sobre o script — por exemplo, faltou declarar `--time`, ou você pediu GPU mas o script não usa nenhuma.
- **No término:** relatório de eficiência — quanto de CPU e memória o job realmente usou, com sugestões concretas (o `--mem` ideal para a próxima submissão, já calculado).

Os e-mails são informativos, não punitivos. Ler e aplicar as sugestões melhora o tempo de fila de todo mundo, inclusive o seu.

8.4 Um diretório por cálculo

⚠ **Atenção:** Nunca rode dois cálculos ao mesmo tempo no **mesmo diretório** quando o programa grava arquivos de nome fixo (o VASP sempre escreve OUTCAR, CONTCAR, WAVECAR, CHGCAR...). Os dois escrevem nos mesmos arquivos e **ambos os resultados ficam inválidos** — mesmo com a eficiência de CPU parecendo ótima, porque os processadores estão de fato ocupados, produzindo lixo. O cluster detecta e avisa por e-mail, mas não cancela: escolher qual preservar é decisão de quem submeteu.

Para evitar, use um diretório por cálculo. Uma forma prática é criá-lo no próprio script:

```
mkdir -p caso_${SLURM_JOB_ID} && cd caso_${SLURM_JOB_ID}
```


9. Containers com Apptainer

9.1 Por que Apptainer e não Docker

O cluster oferece o **Apptainer** (sucessor do Singularity, padrão em HPC) em todos os nós. Com ele você roda qualquer imagem — inclusive as do Docker Hub — **sem precisar de root** e sem sair do controle do Slurm: o container executa com o seu usuário, dentro do seu job, contando na sua memória e CPU reservadas.

👉 **Importante:** Docker **não é** ferramenta de usuário neste cluster. Containers do Docker escapam da contabilidade do Slurm e do limite de memória do job, o que equivale a rodar fora da fila (seção 8.1). Tudo que roda em Docker roda em Apptainer — inclusive a mesma imagem.

9.2 Usando imagens prontas iniciante

```
# baixar uma imagem e converter para .sif (um arquivo unico)
apptainer pull docker://ubuntu:24.04
# -> cria ubuntu_24.04.sif

# abrir um shell dentro dela
apptainer shell ubuntu_24.04.sif

# executar um comando
apptainer exec ubuntu_24.04.sif python3 --version

# tambem funciona direto, sem pull antes:
apptainer exec docker://tensorflow/tensorflow:latest python -c "import tensorflow"
```

Seu home fica visível dentro do container automaticamente — os arquivos que o programa gravar são seus, no lugar de sempre. Outros diretórios podem ser montados com `--bind /caminho`.

💡 **Dica:** O cache de imagens fica em `~/.apptainer/cache` e cresce rápido. De tempos em tempos: `apptainer cache clean`.

9.3 Containers com GPU

Basta a flag `--nv`, que injeta os drivers NVIDIA do nó:

```
apptainer exec --nv docker://nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

9.4 Construindo sua própria imagem avançado

Você pode construir imagens a partir de uma receita (`.def`), sem pedir nada ao administrador:

```
# receita.def
Bootstrap: docker
From: ubuntu:24.04

%post
    apt-get update && apt-get install -y python3 python3-pip
    pip3 install --break-system-packages numpy scipy

%runscript
    python3 "$@"
```

```
apptainer build minha.sif receita.def
apptainer run minha.sif script.py
```

Durante o `%post` você "é root" dentro do container (fakeroot) — pode usar `apt-get` à vontade. A imagem final e tudo que ela gravar no seu home continuam com o seu usuário.

⚠ **Atenção:** Imagens de base **Alpine** (musl) precisam de uma flag a mais: `apptainer build --ignore-fakeroot-command minha.sif receita.def`. Bases glibc (ubuntu, debian, rockylinux) constroem sem flag nenhuma.

9.5 Containers dentro de jobs

É o uso normal: o container respeita o cgroup do job, então `--mem` e CPUs valem para ele como para qualquer programa.

```
#!/bin/bash
#SBATCH --job-name=container
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=12:00:00

apptainer exec --nv ~/imagens/pytorch.sif python -u treino.py
```

10. Armazenamento e boas práticas

Caminho	O que é	Uso
/home/usuario	Sua home (ZFS, 83 TB no total), visível em todos os nós	Código, entradas, resultados
/opt	Softwares instalados (somente leitura para usuários)	—
/tmp (de cada nó)	Disco local do nó, não compartilhado	Arquivos temporários durante o job — apague ao final

Organização de arquivos

Um diretório por projeto, e dentro dele um por cálculo. Algo como:

```
~/projeto_perovskitas/  
├─ estruturas/  
├─ calculo_001/          # INCAR, POSCAR, KPOINTS, POTCAR, job.sh  
├─ calculo_002/  
└─ analise/              # scripts e graficos
```

Boas práticas

- **Nunca** execute jobs pesados no nó de login.
- Estime o tempo e coloque um `--time` realista — nem apertado (o job é morto ao estourar), nem exagerado (atrapalha o backfill).
- Cancele com `scancel` os jobs que você percebeu que não precisa mais. Isso libera a fila imediatamente.
- A home **não é infinita** nem tem backup automático de tudo: mantenha cópia externa do que é insubstituível e apague o que já cumpriu seu papel (WAVECAR antigo, checkpoint de treino concluído, cache de imagens).
- Milhões de arquivos pequenos degradam o sistema para todos — prefira agregar (`tar` , `HDF5`, `zarr`).
- Veja seu consumo com `du -sh ~/*` (é uma operação pesada; rode com moderação).

11. Perguntas frequentes

Meu job está pendente há muito tempo. É normal?

Veja o motivo em `squeue --me`. `Resources` ou `Priority`: a fila está cheia, aguarde.

`QOSMaxCpuPerUserLimit`: você atingiu seu teto de CPUs; ele entra quando seus outros jobs terminarem. Jobs com `--time` curto e declarado entram mais fácil (backfill).

Meu job morreu com `OUT_OF_MEMORY`.

Pediu menos memória do que usou. `sacct -j JOBID -o JobID,State,MaxRSS` mostra o pico por tarefa; aumente o `--mem` (o e-mail da IlumA sobre o job costuma já trazer o valor sugerido) e resubmeta.

`sbatch: error: Memory specification can not be satisfied`

Você pediu mais memória do que qualquer nó tem disponível. Acima do teto do maior nó é preciso repartir o cálculo (MPI) em vez de aumentar o `--mem`.

Meu job falhou na hora, sem gerar o arquivo `slurm-*.out`.

Quase sempre é o diretório de trabalho: se o caminho de onde você submeteu não existir ou estiver inacessível no nó, o Slurm falha antes de conseguir criar o arquivo de saída. Confira se você está numa pasta dentro de `/home` e tente de novo; se persistir, abra um chamado com o JOBID.

Instalei o Miniconda mas `conda` não é encontrado.

Você provavelmente respondeu "no" à pergunta final do instalador. Rode

```
~/miniconda3/bin/conda init bash
```

, saia e entre de novo.

Recebi um e-mail dizendo que processos meus foram finalizados.

Você rodou cálculo fora da fila (seção [8.1](#)). Nada foi perdido além daquele processo; submeta o mesmo cálculo via `sbatch`.

Como uso dois nós ao mesmo tempo?

O cluster aloca um nó por job. Programas não-MPI (Python comum, a maioria dos scripts) não conseguem usar mais de um nó de qualquer forma. Para varreduras de muitos casos independentes, use um *job array* (seção [7.4](#)) — é mais eficiente e entra na fila mais rápido.

Posso deixar algo rodando depois que eu deslogar?

Jobs do Slurm continuam rodando independentemente do seu login — é o normal. O que morre

ao deslogar são processos soltos no terminal (que, nos nós de cálculo, não são permitidos de toda forma).

O programa X não está instalado. E agora?

Três caminhos, do mais rápido ao mais formal: instalar no seu conda (`pip install`), rodar via Apptainer (`apptainer exec docker://...`), ou abrir chamado pedindo a instalação em `/opt/apps` .

12. Guia rápido de referência

Cheatsheet de Linux

Comando	O que faz
<code>pwd</code>	Mostra o diretório atual
<code>ls -lh</code>	Lista arquivos com detalhes
<code>cd pasta/</code>	Entra na pasta
<code>cd ..</code>	Volta um nível
<code>mkdir nome</code>	Cria diretório
<code>cp a b</code>	Copia <code>a</code> para <code>b</code>
<code>mv a b</code>	Move ou renomeia
<code>rm arquivo</code>	Remove (permanente!)
<code>nano arquivo</code>	Edita arquivo
<code>cat / less</code>	Exibe arquivo / exibe paginado
<code>tail -f arquivo</code>	Acompanha em tempo real
<code>grep "texto" arq</code>	Busca texto no arquivo
<code>scp arq user@host:dir/</code>	Envia arquivo via SSH
<code>Ctrl + C</code>	Cancela o comando atual
<code>Tab</code>	Autocompleta
<code>man comando</code>	Manual do comando

Cheatsheet do Slurm

Comando	O que faz
---------	-----------

<code>sinfo</code>	Estado das filas e nós
<code>squeue</code>	Jobs na fila (todos)
<code>squeue --me</code>	Só os seus jobs
<code>sbatch script.sh</code>	Submeter job
<code>scancel JOBID</code>	Cancelar job
<code>scontrol show job JOBID</code>	Detalhes do job
<code>sacct -j JOBID</code>	Histórico e uso de recursos
<code>srun ... --pty bash</code>	Sessão interativa
<code>sacctmgr show assoc where user=\$USER</code>	Seu limite de CPUs

Cheatsheet de Módulos

Comando	O que faz
<code>module avail</code>	Lista módulos disponíveis
<code>module load nome/versao</code>	Carrega módulo
<code>module list</code>	Módulos carregados
<code>module unload nome</code>	Descarrega módulo
<code>module purge</code>	Descarrega todos
<code>module show nome</code>	Mostra o que o módulo altera

Cheatsheet do Conda / Python

Comando	O que faz
<code>conda activate ilumpy</code>	Ativa o ambiente
<code>conda deactivate</code>	Desativa o ambiente atual
<code>conda env list</code>	Lista todos os ambientes

<code>conda list</code>	Pacotes do ambiente ativo
<code>pip install pacote</code>	Instala pacote no ambiente ativo
<code>pip list</code>	Pacotes instalados via pip
<code>pip show pacote</code>	Informações sobre um pacote
<code>python -u script.py</code>	Executa sem buffer de saída

Cheatsheet do Apptainer

Comando	O que faz
<code>apptainer pull docker://imagem</code>	Baixa e converte para <code>.sif</code>
<code>apptainer exec img.sif cmd</code>	Executa um comando no container
<code>apptainer shell img.sif</code>	Abre um shell no container
<code>apptainer exec --nv ...</code>	Com acesso à GPU
<code>apptainer build img.sif receita.def</code>	Constrói imagem própria
<code>apptainer cache clean</code>	Limpa o cache de imagens

13. Glossário

Ambiente virtual (conda/venv)

Espaço isolado com versão própria do Python e bibliotecas, sem interferir no sistema nem em outros projetos.

Apptainer

Sistema de containers usado em HPC; roda imagens (inclusive do Docker) sem privilégios de root.

Backfill

Estratégia do escalonador que encaixa jobs curtos nos intervalos ociosos, desde que saiba quanto eles duram.

cgroup

Mecanismo do Linux que o Slurm usa para impor os limites de CPU e memória de cada job.

Cluster

Conjunto de computadores interconectados que trabalham como uma única máquina.

conda

Gerenciador de pacotes e ambientes virtuais; base do Miniconda e do Anaconda.

CPU (*Central Processing Unit*)

Processador central; realiza cálculos de propósito geral.

GIL (*Global Interpreter Lock*)

Trava do CPython que impede a execução simultânea de múltiplos *threads* Python; torna scripts comuns essencialmente seriais.

GPU (*Graphics Processing Unit*)

Processador gráfico; realiza cálculos paralelos massivos (aprendizado de máquina, simulações em GPU).

HPC (*High-Performance Computing*)

Computação de alto desempenho.

Job

Tarefa submetida ao Slurm para execução nos nós de trabalho.

Job array

Um único job que gera muitas execuções independentes, numeradas por `$SLURM_ARRAY_TASK_ID`.

Miniconda

Distribuição minimalista do conda; instala o gerenciador de pacotes sem a distribuição completa do Anaconda.

Módulo (*Environment Module*)

Mecanismo para carregar e descarregar softwares e suas dependências dinamicamente.

MPI (*Message Passing Interface*)

Padrão para comunicação entre processos paralelos.

Nó (*node*)

Cada computador individual dentro do cluster.

Nó de login

Ponto de acesso ao cluster (`work0`); não deve ser usado para cálculos pesados.

OOM (*Out Of Memory*)

Encerramento do job por ter ultrapassado a memória reservada.

OpenMP

API para paralelismo de memória compartilhada (multithreading dentro de um processo).

Partição (*partition*)

Fila de jobs com regras específicas de acesso e recursos.

QOS (*Quality of Service*)

Política que limita o uso de recursos por usuário ou grupo.

Slurm

Sistema de gerenciamento de recursos e filas de jobs usado no Heisenberg.

SSH (*Secure Shell*)

Protocolo criptografado para acesso remoto a servidores.

14. Checklist do primeiro uso

- ☐ Obtive minha conta na Ilum/CNPEM.
- ☐ Abri o PowerShell (Windows) ou o terminal (Linux/macOS).
- ☐ Me conectei ao cluster: `ssh usuario@heisenberg.cnpem.br`.
- ☐ Explorei minha pasta home com `ls` e `pwd`.
- ☐ Listei os módulos disponíveis com `module avail`.
- ☐ Instalei o Miniconda na minha home.
- ☐ Criei o ambiente `ilumpy` com Python 3.11.
- ☐ Instalei as bibliotecas científicas com `pip install`.
- ☐ Verifiquei o estado das filas com `sinfo`.
- ☐ Descobri meu limite de CPUs com `sacctmgr show assoc where user=$USER`.
- ☐ Escrevi e submeti meu primeiro job com `sbatch`.
- ☐ Acompanhei o job com `squeue --me` e `tail -f slurm-*.out`.
- ☐ Li o relatório de eficiência que a IlumA me enviou por e-mail.

15. Suporte

- **Problemas, dúvidas, pedidos de software ou de aumento de limite:** abra um chamado na Web Ilum — <https://web-ilum.cnpem.br/chamado>
- **Troca ou recuperação de senha:** <http://172.20.10.15>

Ao reportar um problema com um job, inclua sempre o **número do job** (JOBID), o script de submissão e o arquivo `slurm-JOBID.out` — com isso o diagnóstico quase sempre sai na primeira resposta.

Manual do Cluster Heisenberg · Ilum/CNPEM · agosto de 2026. Sugestões de melhoria deste manual são bem-vindas via chamado.